

Apollo Lunar Module Powered Descent

A Detailed Walkthrough of `apollo_full.py`

Technical documentation

29 July 2026

Contents

1	Overview	3
1.1	Structure of the file	3
2	Problem layout: what the vehicle model actually contains	4
2.1	The augmented state vector	4
2.2	The control vector	4
2.3	Index tables	4
3	Parameters	5
3.1	LMPParams — the vehicle	5
3.1.1	Engine placement — <code>eng_pos(i)</code>	5
3.1.2	RCS geometry — <code>rcs_geometry()</code>	6
3.2	OCPCConfig — the transcription and the constraints	6
3.2.1	Grid choice	6
3.2.2	The glide-slope constraint — why it exists	6
3.2.3	Cost weights	7
3.3	Scenario — the initial condition	7
4	Dynamics	9
4.1	<code>flat_moon_6dof(x, u, lm)</code>	9
4.1.1	1. RCS wrench	9
4.1.2	2. TVC forces and moments, engine by engine	9
4.1.3	3. Actuator dynamics	9
4.1.4	4. Rigid-body equations	10
4.2	Integrators	10
5	Building and solving the NLP	11
5.1	Why scaling is the central trick — <code>nlp_scales</code>	11
5.2	<code>solve_landing</code> step by step	11
5.2.1	Augmenting the scenario	11
5.2.2	Decision variables and transcription	11
5.2.3	Constraints	12
5.2.4	Objective	12
5.2.5	Warm start	13

5.2.6	Solver options	13
5.3	The engine-off phase — <code>cutoff_freefall</code>	14
6	Reporting	15
7	The figures	16
7.1	<code>states.png</code> — <code>plot_states</code>	16
7.2	<code>controls.png</code> — <code>plot_controls</code>	16
7.3	<code>actuators.png</code> — <code>plot_actuators</code>	16
7.4	<code>thrusters.png</code> — <code>plot_thrusters</code>	16
7.5	<code>trajectory_with_axes.png</code> — <code>plot_trajectory_with_axes</code>	16
8	Execution flow	18
9	Modelling assumptions and their consequences	19
10	Quick reference	20
10.1	Dimensions	20
10.2	Key tuning knobs	20

1 Overview

`apollo_full.py` is a single, self-contained script that plans and visualises a lunar landing. It does exactly one thing, once:

1. It builds an **open-loop optimal control problem (OCP)** describing the powered descent of an Apollo-class Lunar Module (LM) from an initial state roughly 1 km above and 320 m downrange of the landing pad, down to a low *contact* altitude directly over the pad.
2. It **transcribes** that continuous-time problem into a finite-dimensional nonlinear program (NLP) using *direct multiple shooting* with a fixed-step Runge–Kutta integrator, and solves it once with IPOPT through CasADi.
3. It then simulates a short **engine-off ballistic settle** from the contact altitude to the ground, because the real Descent Propulsion System (DPS) is cut at probe contact rather than throttled to zero at touchdown.
4. Finally it prints a landing report and renders five figures: state histories, control histories, actuator command-vs-actual traces, the 16 individual RCS thruster firings, and a 3-D trajectory with body axes and the glide-slope cone drawn in.

There is **no feedback controller and no closed loop**. The script produces a reference trajectory and the open-loop commands that generate it. Everything downstream (plots, reports) is a presentation of that single solve.

The three external dependencies are `casadi` (symbolic modelling + NLP interface), `numpy`, and `matplotlib`.

1.1 Structure of the file

Section	Lines (approx.)	Content
0. Problem layout	18–54	State/control vector sizes and index tables
1. Parameters	57–213	<code>LMPParams</code> , <code>OCPCConfig</code> , <code>Scenario</code> dataclasses
2. Dynamics	216–316	6-DOF equations of motion, RK4/RK2 integrators
3. OCP build & solve	319–528	Scaling, transcription, constraints, cost, solve, free-fall
4. Landing report	531–578	Terminal-state and cut-off console reports
5. State & control plots	581–740	Four 2-D figure generators
6. Trajectory snapshot	743–832	3-D trajectory figure with body axes
Main	839–859	Drives the whole pipeline

2 Problem layout: what the vehicle model actually contains

2.1 The augmented state vector

The script does not model a point mass or even a bare rigid body. It models a rigid body **plus the internal dynamics of every actuator**, so the optimiser cannot command physically impossible instantaneous thrust or gimbal changes.

The state is 22-dimensional, $n_x = 12 + 5N_{\text{eng}}$ with $N_{\text{eng}} = 2$:

$$x = \left[\underbrace{x_E, y_E, z_E}_{\text{position}}, \underbrace{u, v, w}_{\text{body velocity}}, \underbrace{\phi, \theta, \psi}_{\text{Euler}}, \underbrace{p, q, r}_{\text{body rates}}, \underbrace{T_i, \delta_{p,i}, \dot{\delta}_{p,i}, \delta_{y,i}, \dot{\delta}_{y,i}}_{\text{engine } i=0,1} \right]^\top$$

- **Position** (x_E, y_E, z_E) is in an Earth-fixed (here: moon-fixed) NED-like frame with z_E **positive down**. Altitude is therefore $-z_E$, which is why the plotting code writes `alt = -xs[2, :]` everywhere.
- **Velocity** (u, v, w) is expressed in the **body** frame, not the inertial frame. This is the classical flight-dynamics convention and is what makes the translational equations contain the $\omega \times v$ Coriolis terms.
- **Attitude** is a 3-2-1 Euler triple. This is fine here because the configuration keeps $|\theta| \leq 45^\circ$, well away from the gimbal-lock singularity at $\theta = \pm 90^\circ$ where $1/\cos \theta$ blows up in the kinematic equation.
- **Actuator states**: per engine, the *actual* thrust T_i , the *actual* pitch and yaw gimbal angles $\delta_{p,i}, \delta_{y,i}$, and their rates.

2.2 The control vector

The control is also 22-dimensional, $n_u = 3N_{\text{eng}} + 16$:

$$u = \left[\underbrace{T_{c,i}, \delta_{p,c,i}, \delta_{y,c,i}}_{\text{engine } i=0,1}, \underbrace{f_1, \dots, f_{16}}_{\text{RCS thrusters}} \right]^\top$$

The DPS entries are **commands**, not applied values — they are the setpoints fed to the actuator lag models. The 16 RCS entries are individual thruster force magnitudes, each constrained to $[0, F_{\text{rcs,per}}]$ (a chemical thruster fires one way or not at all).

2.3 Index tables

Lines 47–54 build the index lists once, at import time, rather than hard-coding magic numbers:

```

IDX_T, IDX_DP, IDX_DPD, IDX_DY, IDX_DYD = [
    [N_RIGID + i * N_ACT_PER_ENG + j for i in range(N_ENG)] for j in range(5)]

```

So `IDX_T == [12, 17]` (engine 0's thrust sits at state 12, engine 1's at 17), `IDX_DP == [13, 18]`, and so on. The same trick gives `IDX_U_T == [0, 3]` for the control vector. `IDX_ACT_ALL` is the sorted union, used to zero every actuator state at engine cut-off. The benefit is that changing `N_ENG` to 1 or 4 reconfigures the entire model — dynamics, bounds, scaling, and plots — with no other edits.

3 Parameters

3.1 LMParams — the vehicle

Field	Value	Meaning
mass	7711 kg	Vehicle mass, held constant (see caveats)
Ixx, Iyy, Izz	5368, 5368, 5040 kg · m ²	Principal inertias; products of inertia assumed zero
g_moon	1.625 m/s ²	Lunar surface gravity
T_max, T_min	45 040 N, 4 560 N	Total DPS thrust envelope (both engines)
gimbal_max	6°	Gimbal deflection limit
dz_eng	2.5 m	Engine plane below the CG, body +z
y_eng	1.5 m	Lateral half-spacing of the two engines
tau_T	0.4 s	Thrust first-order lag time constant
gimbal_wn, gimbal_zeta	4 rad/s, 0.7	Gimbal actuator natural frequency and damping
F_rcs_per	445 N	Max thrust of a single RCS thruster
n_quads	4	RCS quads → 16 thrusters total
rcs_arm	1.7 m	Quad radius from the centreline

The derived properties divide the *nominal* envelope by the engine count, so $T_{\max, \text{eng}} = 22\,520$ N and the two engines together reproduce the single-engine envelope exactly. $\mathbf{T}_{\text{hover}} = mg_{\text{moon}} = 12\,530$ N is the total thrust needed to hold altitude at zero attitude.

3.1.1 Engine placement — `eng_pos(i)`

```
off = 0.0 if n_eng == 1 else y_eng * (2.0 * i / (n_eng - 1) - 1.0)
return np.array([0.0, off, dz_eng])
```

Engines are spread evenly along the body y -axis from $-y_{\text{eng}}$ to $+y_{\text{eng}}$: for $N_{\text{eng}} = 2$, engine 0 sits at $y = -1.5$ m and engine 1 at $y = +1.5$ m, both 2.5 m below the CG. Two consequences follow:

- Because the moment arm is $\pm y_{\text{eng}}$ and the thrust is predominantly along $-z_B$, **differential throttling produces a direct roll moment** $L \approx (T_1 - T_0) y_{\text{eng}}$. This is a control authority the single-engine model simply does not have.
- At equal thrust the two roll moments cancel exactly, so trimmed flight is identical to the single-engine case. The extra authority is free.

3.1.2 RCS geometry — `rsc_geometry()`

Four quads sit at azimuths 45° , 135° , 225° , 315° at radius 1.7 m in the body x - y plane. Each quad carries four thrusters: one firing $+z$ (down), one $-z$ (up), and two tangential ($\pm$) in the plane. For thruster i at position r_i firing along unit direction d_i , the wrench it produces is $[d_i; r_i \times d_i]$, so stacking these as columns gives the 6×16 allocation matrix B with

$$\begin{bmatrix} F_{\text{rsc}} \\ M_{\text{rsc}} \end{bmatrix} = Bf, \quad f_i \in [0, 445 \text{ N}].$$

Every thruster produces a **coupled** force and moment — there is no pure torque jet. This is why the plotting code refuses to draw axis-aligned bound lines on the net RCS wrench: the reachable set is the image of a box under B , a zonotope, not a box.

The `B_rsc` property caches the CasADi DM version. This matters: the dynamics function is called 4 times per RK4 step across 80 steps while the symbolic graph is built, and rebuilding the NumPy geometry each time was measurable overhead.

3.2 OCPCConfig — the transcription and the constraints

Field	Value	Role
<code>N, dt</code>	80, 1.0 s	80-step grid $\rightarrow$ 80 s powered-descent horizon
<code>integrator</code>	'rk4'	Or 'rk2' (Heun)
<code>V_max</code>	60 m/s	Per-axis body-velocity limit
<code>euler_max</code>	45°	Roll/pitch/yaw limit
<code>omega_max</code>	$10^\circ/\text{s}$	Body-rate limit
<code>h_contact</code>	1.0 m	Altitude at which the DPS is cut
<code>glide_slope</code>	30°	Approach-cone half-angle measured up from horizontal
<code>Qs, Qf</code>	see below	Stage / terminal state weights (12 entries)
<code>Rw, Rd</code>	see below	Control and control-rate weights (22 entries)

3.2.1 Grid choice

The comment records that the grid was coarsened from 200 steps at 0.5 s. The reasoning: IPOPT's cost per iteration is dominated by factorising the KKT system, which grows roughly quadratically in N . At the same time $dt = 1$ s is about the coarsest step the gimbal actuator tolerates — with $\omega_n = 4$ rad/s its period is ≈ 1.6 s, so a 1 s step barely resolves the second-order response. Going coarser would alias the actuator dynamics.

3.2.2 The glide-slope constraint — why it exists

This is the most consequential single line of configuration. The constraint is

$$\tan(\gamma)\sqrt{x_E^2 + y_E^2} \leq -z_E \quad (\text{altitude}), \quad \gamma = 30^\circ,$$

i.e. the vehicle must remain inside an upward cone with its apex at the pad.

Without it, the solution is pathological, and the docstring explains exactly why. The stage cost penalises distance to the target at *every* node, and the target sits at h_{contact} . So the cheapest thing the optimiser can do is **dive at the velocity limit** to 1 m altitude as fast as possible, accumulate a big horizontal overshoot (it cannot decelerate laterally that fast), skim the ground ~58 m downrange, and then crawl back to the pad and hover. That is the sharp hook that used to appear in the 3-D plot. The cone rules that whole family out: *being low is only permitted when you are already close*, which forces a straight-in approach.

Feasibility is checked in the comment: the initial state is 323 m downrange at 1000 m altitude, which is $\arctan(1000/323) \approx 72^\circ$ — comfortably inside a 30° cone, so the initial state is admissible with wide margin.

3.2.3 Cost weights

```
Qs = [20, 20, 30, 60, 60, 60, 20, 20, 1, 30, 30, 30]
Qf = [5000, 5000, 8000, 4000, 4000, 4000, 6000, 6000, 400, 10000, 10000, 10000]
Rw = [1e-7, 8.0, 8.0] * 2 + [5e-4] * 16
Rd = [1e-6, 4000.0, 4000.0] * 2 + [1e-3] * 16
```

Reading these as engineering intent:

- $Q_f \gg Q_s$ by two to three orders of magnitude — the terminal state is what actually matters; the stage cost is mostly there to shape the path.
- Yaw ψ is weighted at 1 (stage) and 400 (terminal) versus 20/6000 for roll and pitch: heading is nearly irrelevant for a landing, attitude in pitch/roll is not.
- R_w on thrust is 10^{-7} — essentially free, because thrust magnitudes are $\sim 10^4$ N and squaring gives 10^8 ; without the tiny weight thrust would dominate the entire cost. The gimbal weight of 8.0 acts on radians, so a full 6° deflection contributes $8 \times 0.105^2 \approx 0.09$ — small but enough to discourage aimless gimbaling.
- R_d is the **rate** penalty on $\Delta u = u_k - u_{k-1}$, and it is heavy (4000) on the gimbals. This is the term that damps gimbal chatter; without it the optimiser happily bangs the gimbal from limit to limit between nodes because a zero-order-hold discretisation makes that free.
- RCS commands are penalised toward **zero** (not toward a trim value), which makes RCS usage a minimum-fuel term.

3.3 Scenario — the initial condition

$$x_0 : \quad (x_E, y_E, z_E) = (300, 120, -1000) \text{ m}, \quad (u, v, w) = (-8, -2, 5) \text{ m/s}, \\ (\phi, \theta, \psi) = (2^\circ, -3^\circ, 10^\circ), \quad (p, q, r) = (0.3, -0.4, 0.1)^\circ/\text{s}.$$

So: 1 km up, 323 m horizontally out, already descending at 5 m/s and moving back toward the pad at 8 m/s, with a small attitude and rate offset to make the problem non-trivial. The target $\mathbf{x}_{\text{target}}$ is the 12-vector of zeros, later overridden in z to $-h_{\text{contact}}$.

Scenario also carries the engine-out fault condition:

```
failed_eng: tuple = ()          # indices of engines dead for the whole horizon
```

A listed engine has all five of its actuator states and all three of its commands pinned to zero, so it contributes neither force nor moment, and the surviving engines each carry $T_{\text{hover}}/n_{\text{live}}$ of trim instead of $T_{\text{hover}}/n_{\text{eng}}$. The dynamics need no change — a zero thrust state produces a zero wrench automatically. Empty tuple is the nominal case and reproduces the healthy solution exactly. See the engine-out case study for what this reveals about the two-engine layout.

4 Dynamics

4.1 flat_moon_6dof(x, u, lm)

This returns $\dot{x} \in \mathbb{R}^{22}$ symbolically (CasADi expressions), so it can be differentiated to arbitrary order by the NLP solver. “Flat moon” means a flat, non-rotating body: no oblateness, no Coriolis from planetary rotation, constant gravity.

4.1.1 1. RCS wrench

```
f_rcs = u[N_U_DPS : N_U_DPS + lm.n_rcs]
wrench = lm.B_rcs @ f_rcs
```

One matrix product turns 16 scalar firings into (F_{rx}, F_{ry}, F_{rz}) and (L_r, M_r, N_r) .

4.1.2 2. TVC forces and moments, engine by engine

For each engine, with actual (not commanded) gimbal angles and $T_{\text{eff}} = \eta_i T$:

$$T_x = T_{\text{eff}} \sin \delta_p, \quad T_y = -T_{\text{eff}} \sin \delta_y \cos \delta_p, \quad T_z = -T_{\text{eff}} \cos \delta_p \cos \delta_y.$$

$\eta_i = \text{lm.eta_of}(i)$ is the engine’s **thrust efficiency**, 1.0 unless overridden in **thrust_eff_eng**. Only ηT becomes force; the actuator dynamics below still act on the full T , so a partially-efficient engine burns propellant for T while the vehicle only feels ηT — the shortfall is lost power, not saved fuel. See the thrust-efficiency case study.

At zero deflection this gives $(0, 0, -T)$: thrust along body $-z$, i.e. *up*, consistent with the z -down body frame. The moment about the CG is $M_i = r_i \times F_i$ with $r_i = (0, y_i, 2.5)$, expanded component-wise:

$$L += y_e T_z - z_e T_y, \quad M += z_e T_x - x_e T_z, \quad N += x_e T_y - y_e T_x.$$

The $y_e T_z$ term in L is precisely the differential-throttle roll authority discussed earlier.

4.1.3 3. Actuator dynamics

Thrust follows a first-order lag toward its command,

$$\dot{T}_i = \frac{T_{c,i} - T_i}{\tau_T}, \quad \tau_T = 0.4 \text{ s},$$

and each gimbal axis is a second-order servo,

$$\ddot{\delta} = \omega_n^2 (\delta_c - \delta) - 2\zeta\omega_n \dot{\delta}, \quad \omega_n = 4 \text{ rad/s}, \quad \zeta = 0.7.$$

ω_n and ζ are read **per engine** via **lm.wn_of(i)** / **lm.zeta_of(i)**, which fall back to the nominal **gimbal_wn** / **gimbal_zeta** unless overridden in **gimbal_wn_eng** / **gimbal_zeta_eng**. That is what lets one engine carry a degraded actuator while the other stays healthy; the gimbal-rate scale factors

in `nlp_scales` follow the same per-engine values, since a sluggish actuator’s rate state is genuinely an order of magnitude smaller. See the degraded-gimbal case study.

These six (per engine: $1 + 2 \times 2$) derivatives are appended to `act_dot` and returned at the tail of $\dot{x}$. This is why the actuator plots can show “commanded vs actual” as genuinely different traces.

4.1.4 4. Rigid-body equations

Attitude DCM (3-2-1, body $\rightarrow$ Earth):

$$C_{E/B} = \begin{bmatrix} c\theta c\psi & s\phi s\theta c\psi - c\phi s\psi & c\phi s\theta c\psi + s\phi s\psi \\ c\theta s\psi & s\phi s\theta s\psi + c\phi c\psi & c\phi s\theta s\psi - s\phi c\psi \\ -s\theta & s\phi c\theta & c\phi c\theta \end{bmatrix}$$

Translational kinematics: $\dot{r}_E = C_{E/B} [u, v, w]^\top$.

Translational dynamics (body frame, so the transport term appears):

$$\dot{v}_b = \begin{bmatrix} rv - qw \\ pw - ru \\ qu - pv \end{bmatrix} + g_{\text{moon}} \begin{bmatrix} -s\theta \\ s\phi c\theta \\ c\phi c\theta \end{bmatrix} + \frac{1}{m} \begin{bmatrix} F_x \\ F_y \\ F_z \end{bmatrix}$$

The middle term is gravity resolved into the body frame — it is the third *column* of $C_{E/B}^\top$ acting on $(0, 0, g)$, which is the third *row* of $C_{E/B}$.

Euler kinematics:

$$\dot{\phi} = p + (qs\phi + rc\phi) \tan \theta, \quad \dot{\theta} = qc\phi - rs\phi, \quad \dot{\psi} = \frac{qs\phi + rc\phi}{\cos \theta}$$

Rotational dynamics (Euler’s equations, diagonal inertia):

$$\dot{p} = \frac{L - (I_{zz} - I_{yy})qr}{I_{xx}}, \quad \dot{q} = \frac{M - (I_{xx} - I_{zz})pr}{I_{yy}}, \quad \dot{r} = \frac{N - (I_{yy} - I_{xx})pq}{I_{zz}}$$

4.2 Integrators

`rk4_step` is classical fourth-order Runge–Kutta with a **zero-order hold** on the control: u is held constant across all four stages, which is exactly the right model for a digital autopilot issuing one command per grid interval.

`rk2_step` is Heun’s method (explicit trapezoid), two dynamics evaluations instead of four. The docstring is honest about the trade: it halves the per-step constraint graph, but in practice needs more IPOPT iterations here, so RK4 wins end-to-end. `INTEGRATORS` is a dispatch dictionary keyed by `cfg.integrator`.

5 Building and solving the NLP

5.1 Why scaling is the central trick — `nlp_scales`

The raw problem spans about seven decades: thrust is $\sim 4.5 \times 10^4$ N sitting in the same vector as gimbal rates of $\sim 10^{-2}$ rad/s. An interior-point method forms and factorises a KKT matrix built from these quantities; if their magnitudes differ that wildly the matrix is badly conditioned, IPOPT’s step computation is inaccurate, and it takes hundreds of tiny steps.

The fix is a diagonal change of variables. `nlp_scales` returns per-variable scale factors S_x, S_u chosen as the *natural magnitude* of each quantity — usually its own bound:

Variable	Scale
position	100 m
body velocity	10 m/s
attitude	<code>euler_max</code> = 45°
body rates	<code>omega_max</code> = 10°/s
thrust	<code>T_max_eng</code> = 22 520 N
gimbal angle	<code>gimbal_max</code> = 6°
gimbal rate	$\omega_n \cdot \delta_{\max}$
RCS command	<code>F_rcs_per</code> = 445 N

IPOPT then works with $\tilde{X}, \tilde{U}$ where $X = S_x \tilde{X}$ and $U = S_u \tilde{U}$, so box constraints sit at roughly ± 1 everywhere.

The measured effect, recorded in the comments: **1803 iterations** $\rightarrow$ **85** at $N = 80$, and total solve time ≈ 690 s $\rightarrow$ 34 s (together with `detect_simple_bounds`), for a solution within 0.03 % of the unscaled one’s cost.

5.2 solve_landing step by step

5.2.1 Augmenting the scenario

The 12-state scenario and weights are padded out to 22:

```
act_x0 = np.array([lm.T_hover_eng, 0, 0, 0, 0] * lm.n_eng)
x_targ[2] = -cfg.h_contact
Qs = np.concatenate([cfg.Qs, np.zeros(N_ACT)])    # actuator states unweighted
```

Each engine starts at its share of hover thrust with gimbals centred and still. Actuator states get **zero** weight — they are free internal variables, not things we want driven to a particular value. Critically, the target altitude is set to $-h_{\text{contact}}$, not zero: the powered phase is asked to settle at 1 m over the pad, not on the ground.

5.2.2 Decision variables and transcription

```
opti = ca.Opti()
Xv = opti.variable(nx, cfg.N + 1)    # 22 x 81 scaled states
Uv = opti.variable(nu, cfg.N)        # 22 x 80 scaled controls
```

That is $22 \times 81 + 22 \times 80 = 3542$ decision variables. This is **direct multiple shooting**: the state at every node is an independent variable, and the dynamics enter as equality (defect) constraints

$$\tilde{X}_{k+1} = \frac{1}{S_x} \Phi(S_x \tilde{X}_k, S_u \tilde{U}_k, \Delta t)$$

where Φ is one RK4 step. Dividing the defect by S_x is the same scaling argument applied to the *rows* of the constraint Jacobian rather than the columns.

Multiple shooting (as opposed to single shooting) is what makes this tractable: each defect couples only two adjacent nodes, so the Jacobian is block-banded and sparse, and the solver never has to propagate sensitivities across all 80 steps.

5.2.3 Constraints

Initial condition. $\tilde{X}_{:,0} = x_0/S_x$.

Glide slope. Written squared to stay differentiable:

```
opti.subject_to(tan2 * (xk[0]**2 + xk[1]**2) <= xk[2]**2)
```

The comment explains both halves of this choice. Squaring avoids the kink of $\sqrt{x^2 + y^2}$ at the origin — which is precisely where the vehicle ends up, so a non-smooth constraint there would be actively harmful. Squaring is *exact* rather than a relaxation only because $z_E \leq -h_{\text{contact}} < 0$ is enforced separately, so the spurious downward branch of the cone is unreachable. All three components are divided by the same position scale so the Jacobian row is $O(1)$.

Box bounds on every node: velocity, Euler angles, body rates, per-engine thrust and gimbal states, plus the altitude floor $\tilde{X}_{2,k} \leq -h_{\text{contact}}/S_{x,2}$.

The altitude floor is not cosmetic. Because the cone’s apex is at the pad, a trajectory can otherwise ride the cone surface down to nearly 0 m and then have to climb *back up* to the 1 m target — a small bounce that the cone alone does not eliminate. Flooring the altitude removes it.

Control bounds on every interval: per-engine thrust within $[T_{\text{min,eng}}, T_{\text{max,eng}}]$, gimbal commands within $\pm 6^\circ$, and each RCS thruster in $[0, 445]$ N — the one-sided bound that encodes “a thruster cannot pull”.

Note the thrust lower bound: the engine can never be commanded below $4\,560/2$ N. A real DPS cannot be throttled to zero and re-lit, and this is how that shows up in the model. It is also why a separate cut-off phase is needed at all.

5.2.4 Objective

$$J = \sum_{k=0}^{N-1} \left[\|x_k - x_{\text{targ}}\|_{Q_s}^2 + \|u_k - u_{\text{ref}}\|_{R_w}^2 \right] + \sum_{k=1}^{N-1} \|u_k - u_{k-1}\|_{R_d}^2 + \|x_N - x_{\text{targ}}\|_{Q_f}^2$$

with $u_{\text{ref}} = [T_{\text{hover,eng}}, 0, 0]$ per engine and 0 for all RCS channels. Three distinct mechanisms:

1. **Tracking**, quadratic to the target at every node, heavily weighted at the terminal node.
2. **Effort**, measured *relative to hover trim* for thrust (so holding hover costs nothing) and relative to zero for gimbals and RCS.

3. **Rate**, penalising step-to-step command changes. This is what makes the commanded traces in the actuator plot smooth rather than saw-toothed.

Note the cost is evaluated on the **physical xs/us**, not the scaled variables, so the problem being solved is genuinely unchanged by scaling.

5.2.5 Warm start

```
lam = np.linspace(0, 1, N + 1)
X_init = x0[:,None] + (x_targ - x0)[:,None] * lam[None,:]
X_init[IDX_T, :] = lm.T_hover_eng
U_init[IDX_U_T, :] = lm.T_hover_eng
```

A straight line in state space from start to target, with one correction: the actuator states are *held* at hover thrust and centred gimbals rather than being ramped toward the (zero-weighted, zero-valued) target. Ramping thrust toward zero would start the solver outside the thrust box and waste iterations just restoring feasibility.

5.2.6 Solver options

```
opts = {'expand': True, 'detect_simple_bounds': True,
        'ipopt.max_iter': 5000, 'ipopt.tol': 1e-6,
        'ipopt.acceptable_tol': 1e-4, 'ipopt.acceptable_iter': 15,
        'ipopt.mu_strategy': 'adaptive',
        'ipopt.print_level': 3, 'print_time': True}
```

- **expand: True** rewrites the problem graph from CasADi's MX (matrix-valued nodes) to SX (scalar nodes). Every operation used here is SX-compatible, and the payoff measured was roughly 2× faster evaluation of cost, constraints, and derivatives, at the cost of a longer one-time build.
- **detect_simple_bounds: True** is described in the comments as the single biggest per-iteration win. The roughly 2 600 `opti.bounded(...)` calls above are *structurally* just variable bounds, but `Opti` files them into the general constraint vector g . This flag recognises them and moves them into `lbx/ubx`, where IPOPT handles them inside the barrier term instead of carrying 2 600 extra rows through every MUMPS factorisation. This is precisely why all the bounds are written on the *scaled* variables — a bound like $-V_{\max}/Sx[j] \leq Xv[j,k] \leq V_{\max}/Sx[j]$ is a plain bound on a variable; had the code written $-V_{\max} \leq Sx[j]*Xv[j,k]$ it would be an affine expression and the detection would fail.
- **Exact Hessian** (IPOPT's default) rather than L-BFGS: the comment records that limited-memory quasi-Newton simply failed to converge on this NLP. Second derivatives cost more per iteration but the iteration count collapses.
- **tol: 1e-6** can stay tight because iterations are now cheap — the comment notes 1e-6 costs one extra iteration over 1e-4. The **acceptable_*** criteria remain purely as a fallback so a hard scenario terminates at a good feasible point instead of grinding to **max_iter**.
- **mu_strategy: adaptive** lets IPOPT adjust the barrier parameter heuristically rather than following a fixed monotone schedule; usually faster on nonlinear problems like this.

`OCPCConfig.max_iter` feeds `ipopt.max_iter`. The default of 5000 only ever binds on a pathological problem — a feasible scenario here converges in 100–200 iterations — so it is worth lowering when the point of a run is to establish *infeasibility* quickly rather than to find a solution.

The function returns `sol.value(Xv) * Sx[:,None]` and the equivalent for `Uv` — i.e. everything downstream sees **physical units**.

If IPOPT gives up, the solve raises `SolveFailure`. It subclasses `RuntimeError` (so existing handling still catches it) but carries the last iterate in physical units as `.Xs / .Us`, recovered from `opti.debug`. A failed run can therefore still be plotted and inspected rather than vanishing into an exception — which is how the engine-out case study visualises a diverging trajectory.

5.3 The engine-off phase — `cutoff_freefall`

```
x[IDX_ACT_ALL] = 0.0          # hard cut of every engine, gimbals zeroed
u0 = np.zeros(N_U_DPS + n_rcs) # no DPS command, no RCS
```

Starting from the powered terminal state, thrust and gimbals are instantly zeroed and the *same* full 6-DOF dynamics are integrated numerically — with `ca.DM` inputs, so `rk4_step` runs as ordinary numerics rather than symbolics — at a fine $dt = 0.02$ s until z_E reaches 0.

The final sub-step is linearly interpolated so the last sample sits exactly on the ground:

```
frac = -x[2] / (x_next[2] - x[2])
x = x + frac * (x_next - x)
```

The `max_t = 8.0` guard prevents an infinite loop if the vehicle happens to be ascending at cut-off.

Why do this at all rather than just extending the OCP to $z_E = 0$? Three reasons, and the comments give all of them: it mirrors the real DPS shutdown at probe contact; the thrust lower bound means the OCP can never actually reduce thrust to zero; and running the optimiser down to $z = 0$ invites a hover pathology near the ground where the vehicle wants to sit on its own exhaust indefinitely.

6 Reporting

`print_report(Xs)` summarises the terminal state of the *powered* phase: touchdown speed $\|(u, v, w)\|$, position error $\|(x_E, y_E, z_E)\|$, and attitude error $\|(\phi, \theta)\|$ — yaw is deliberately excluded, since heading does not affect landing quality. It then applies a three-part success gate:

$$\|v\| < 2 \text{ m/s}, \quad \|r\| < 15 \text{ m}, \quad \|(\phi, \theta)\| < 5^\circ.$$

`print_cutoff_report(Xc, Xff, tff)` documents the handover: the altitude, descent rate and per-engine thrust at cut-off, then the free-fall duration, final touchdown speed (vertical component called out separately, since that is what the landing gear absorbs), and the horizontal drift accumulated during the unpowered settle.

7 The figures

7.1 `states.png` — `plot_states`

A 4×3 grid: position (with z converted to altitude), body velocity, Euler angles in degrees, and body rates in degrees per second. Red dashed lines mark the active bounds — $\pm V_{\max}$ on the velocity row, $\pm \text{euler_max}$ on the attitude row, $\pm \text{omega_max}$ on the rate row. Reading whether a trace is pinned to its bound tells you immediately which constraint is active and shaping the solution.

7.2 `controls.png` — `plot_controls`

One row per TVC engine showing commanded thrust and the two gimbal commands (with box limits drawn), then two rows giving the **net RCS wrench** reconstructed as Bf — three force components and three moment components. The 16 raw firings would be unreadable here; the wrench is the physically meaningful quantity. As noted, no bound lines are drawn on the wrench because its feasible set is a zonotope, not a box.

All control traces use `ax.step(..., where='post')`, correctly reflecting the zero-order hold assumed by the transcription.

7.3 `actuators.png` — `plot_actuators`

The figure that justifies carrying actuator states at all. Per engine, three panels overlay the **commanded** value (red dashed step) against the **actual** state (solid blue): the first-order thrust lag and the two second-order gimbal responses. The gap between them is the actuator dynamics made visible — lag on thrust, overshoot and settling on the gimbals.

The summary row adds three diagnostics:

1. Both engines' thrust overlaid plus their **total**, against the total $T_{\min}/T_{\max}$ envelope.
2. The **differential thrust** $T_2 - T_1$.
3. The **roll moment from throttling asymmetry alone**, $L = \sum_i y_i (-T_i \cos \delta_{p,i} \cos \delta_{y,i})$, which quantifies exactly how much roll authority the two-engine layout is actually being used for.

7.4 `thrusters.png` — `plot_thrusters`

A 4×4 grid, one panel per RCS thruster, labelled $Qq \cdot Jj$ by quad and jet. Each shows the firing history against the $[0, F_{\text{rcs,per}}]$ bounds. This is where you see the on-off character of the RCS solution and which quads are doing the work.

7.5 `trajectory_with_axes.png` — `plot_trajectory_with_axes`

A 3-D scene containing:

- a translucent ground plane and a red cross-and-star landing pad at the origin, plus a green triangle at the start point;
- the **glide-slope cone** as a shaded orange surface with wireframe. It is deliberately *clipped* to $1.1 \times$ the trajectory's own horizontal extent — drawn in full it would reach $r = h / \tan 30^\circ \approx 1.7$ km at the start altitude and squash the descent into a vertical line;

- the trajectory itself, and at nine evenly spaced frames the **body axes** drawn as an RGB triad via `dcm_eb`, with the z component negated because the plot axis is altitude while the state is z -down.

The main block passes `np.hstack([Xs, Xff])`, so this single figure shows the powered descent **and** the ballistic settle as one continuous path.

Note that `dcm_eb` duplicates, in NumPy, the DCM already built symbolically inside `flat_moon_6dof` — a deliberate duplication so the plotting code has no CasADi dependency.

8 Execution flow

```
if __name__ == '__main__':  
    lm, cfg, sc = LMPParams(), OCPConfig(), Scenario()  
  
    Xs, Us = solve_landing(lm, cfg, sc)      # 1. solve the powered descent  
    print_report(Xs)                        # 2. terminal-state report  
  
    Xff, tff = cutoff_freefall(Xs[:, -1], lm) # 3. engine-off settle  
    print_cutoff_report(Xs, Xff, tff)  
  
    plot_states(Xs, cfg)                    # 4. five figures  
    plot_controls(Us, cfg, lm)  
    plot_actuators(Xs, Us, cfg, lm)  
    plot_thrusters(Us, cfg, lm)  
    plot_trajectory_with_axes(np.hstack([Xs, Xff]), sc, cfg)
```

Data flow in one line:

$$\text{params} \rightarrow \text{NLP} \xrightarrow{\text{IPOPT}} (X_s, U_s) \xrightarrow{\text{cut-off}} X_{ff} \rightarrow \text{reports} + \text{figures}$$

Note the figures are written but `plt.show()` is never called — the script is built for batch/thesis use, saving PNGs at 150 dpi.

9 Modelling assumptions and their consequences

These are the things a reader of the thesis will reasonably ask about.

Constant mass. There is no $\dot{m} = -T/(I_{sp}g_0)$ state. Over an 80 s burn at roughly hover thrust with a DPS $I_{sp} \approx 311$ s, propellant consumption is on the order of 300–350 kg against a 7 711 kg vehicle — a few percent. The inertias are likewise frozen. This is defensible for a planning model and it keeps the dynamics autonomous, but it means the reported trajectory is not a propellant-accurate one, and the cost function does not minimise fuel in the true (mass-flow) sense.

Euler angles. Fine here because $|\theta| \leq 45^\circ$ is enforced. A quaternion formulation would be needed for large-attitude manoeuvres.

Continuous RCS. Real reaction-control thrusters are on/off, pulse-width modulated. Here f_i is a continuous variable in $[0, 445]$ N. This is the standard relaxation — it keeps the NLP smooth instead of turning it into a mixed-integer program — but the commanded profiles are pulse-width *equivalents* rather than realisable firing commands.

Flat, non-rotating moon; no aerodynamics. Both entirely appropriate at this scale.

Nonconvexity. The NLP has multiple local minima, and which one IPOPT finds is sensitive to floating-point details: three runs of the *identical* nominal problem under default multithreaded BLAS gave objectives spread over 1.0 %. Pinning `OMP_NUM_THREADS=1` (plus the OpenBLAS/MKL equivalents) before importing numpy makes a single run bit-reproducible, but it does not make two *different* configurations land in comparable minima — so objective differences between variants of a few percent should not be over-interpreted without a multi-start. This matters whenever the script is used for comparative studies.

Open loop. The output is a reference trajectory plus feedforward commands. There is no disturbance rejection and no navigation error model; a tracking controller would sit downstream of this.

Fixed horizon. $N \cdot dt = 80$ s is fixed, not optimised. The vehicle must reach contact in exactly 80 s. Making the final time a decision variable is the usual next step if that constraint becomes binding.

10 Quick reference

10.1 Dimensions

Quantity	Value
States n_x	22 (12 rigid + 10 actuator)
Controls n_u	22 (6 DPS + 16 RCS)
Grid nodes N	80 (81 state nodes)
Horizon	80 s at $dt = 1$ s
NLP decision variables	3 542
Dynamics defect constraints	$80 \times 22 = 1760$
Reported solve time	≈ 34 s, ≈ 85 IPOPT iterations

10.2 Key tuning knobs

To change...	Edit
Number of engines	N_ENG (everything else resizes)
Approach steepness	OCPConfig.glide_slope
Grid resolution / horizon	OCPConfig.N, OCPConfig.dt
Where the engine is cut	OCPConfig.h_contact
Terminal accuracy vs path smoothness	Qf vs Qs, Rd
Gimbal chatter	Rd entries 1–2 per engine
Solve speed	integrator, N, expand, detect_simple_bounds
Initial condition	Scenario.x0
Engine-out fault	Scenario.failed_eng
Sluggish gimbal on one engine	LMPParams.gimbal_wn_eng, gimbal_zeta_eng
Partial-thrust engine	LMPParams.thrust_eff_eng
Iteration budget	OCPConfig.max_iter